

FPGA-Based FFT Engine for Edge Detection

Using Single-Path Delay-Feedback Architecture

Sreethan Kanagiri , Mithil Damania , Arka Kar , Niraj N. Hariyani
24095045 24095059 24095140 24095144
Department of Electronics Engineering, IIT (BHU), Varanasi, India

Supervisor: Dr. Ankit Arora

Department of Electronics Engineering, IIT (BHU), Varanasi, India

Abstract: Edge detection is a foundational primitive in digital image processing, serving as the critical pre-processing stage for machine vision tasks including autonomous navigation, medical image segmentation, and real-time object recognition. Conventional spatial-domain operators such as Sobel, Prewitt, and Canny are constrained by noise amplification and limited angular resolution. This project presents a robust frequency-domain alternative: a 2-D Fast Fourier Transform (FFT) pipeline that decomposes the input image spectrally, isolates high-frequency boundary content via a dynamically configurable circular high-pass filter (HPF), and reconstructs the spatial edge map via a 2-D Inverse FFT (IFFT). The approach achieves isotropic edge detection highly resilient to sensor noise and illumination variation.

To overcome the $O(N \log_2 N)$ computational burden of real-time 2-D spectral transforms, the full pipeline is hardware-accelerated on a Xilinx Zynq-7000 FPGA using a memory-efficient Single-Path Delay-Feedback (SDF) radix-2 Decimation-In-Frequency (DIF) architecture coded in synthesizable Verilog. The hardware core is integrated into the Zynq-7000 System-on-Chip (SoC), communicating with an ARM Cortex-A9 via AXI4-Stream DMA for seamless hardware-software co-design. Post-implementation results on the XC7Z020 device demonstrate successful timing closure at 70 MHz using only 5 Block RAM tiles and 42 DSP slices. The system achieves a Mean Absolute Pixel Error of 7.26 out of 255 intensity levels (2.85%) relative to a double-precision floating-point reference, validating the Q1.15 convergent-rounding fixed-point arithmetic scheme. Full 1024×1024 edge-map generation completes in under 10 ms per frame, satisfying the 30 fps real-time requirement.

Index Terms: FFT, SDF architecture, FPGA, Zynq-7000, AXI4-Stream DMA, edge detection, high-pass filter, Q1.15 fixed-point arithmetic, hardware-software co-design.

I. INTRODUCTION

Edge detection identifies spatial locations in an image where pixel intensity transitions sharply, corresponding physically to object boundaries, material interfaces, or illumination discontinuities. Reliable edge maps are prerequisite inputs for higher-level vision tasks including object recognition, scene segmentation, depth estimation, and target tracking.

A. Application Domains

The demand for real-time edge detection spans several safety-critical domains. In *medical imaging*, automatic delineation of

organ boundaries in CT or MRI volumes reduces radiologist workload and improves inter-session reproducibility. Tumour margins, ventricular walls, and vascular structures all rely on precise boundary localisation. In *satellite and aerial reconnaissance*, onboard processing must complete before the sensor footprint advances to the next frame, imposing hard latency budgets that preclude ground-based computation. In *autonomous vehicles*, lane markings, pedestrians, and obstacles must be detected reliably under adverse illumination and weather where purely intensity-based heuristics fail. Figure 5 illustrates the complete pipeline output on a natural photograph, demonstrating the quality of the hardware edge map.

B. Limitations of Spatial-Domain Approaches

Classical operators: Sobel, Prewitt, Roberts, and Canny, convolve the image with small derivative kernels (typically 3×3 or 5×5 pixels). Three fundamental limitations constrain their performance.

Noise sensitivity. Derivative filters amplify high-frequency sensor noise alongside genuine edges. The Canny detector mitigates this with a Gaussian pre-blur, but the optimal blur radius is scene-dependent and difficult to tune adaptively [6].

Angular limitation. A 3×3 Sobel kernel provides gradient estimates only along horizontal and vertical axes. Detecting arbitrary orientations requires multiple rotated kernels, increasing cost linearly with the number of target directions.

Locality. Kernel methods examine a small pixel neighbourhood, making them insensitive to slowly varying but globally coherent boundaries such as the diffuse border of a fog-shrouded object or a gradually lit corridor wall.

C. Frequency-Domain Edge Detection

The 2-D Fourier transform addresses all three limitations simultaneously. Sharp edges manifest as high-frequency energy distributed isotropically across the frequency plane, while low-frequency DC energy encodes slow global brightness variations carrying no edge information. A circular high-pass mask that zeroes the DC lobe and passes all frequencies beyond a cut-off radius R selectively removes global illumination trends while

preserving all edge information from every angular orientation equally. Because the mask is parameterised by a single scalar R , it can be updated at run-time by writing a single register, a flexibility unavailable to fixed-kernel spatial filters.

D. Motivation for FPGA Acceleration

A 1024×1024 image requires 2×1024 one-dimensional 1024-point FFTs (one row pass plus one column pass). Even at $O(N \log_2 N)$ complexity this amounts to roughly 2×10^7 complex multiply-accumulate (MAC) operations per frame, prohibitive at 30 fps on a scalar processor without dedicated hardware.

FPGAs overcome this bottleneck through deterministic pipeline throughput and spatial parallelism. A streaming SDF FFT pipeline produces one output sample per clock cycle after an initial fill latency. At 70 Mhz, a 1024-point transform completes in approximately $14.6 \mu\text{s}$, leaving ample margin for the matrix transpose and second FFT pass within the 33 ms frame budget.

E. Contributions

This paper makes the following specific contributions:

- A fully pipelined 1024-point radix-2 DIF SDF FFT engine in synthesisable Verilog with convergent Q1.15 rounding and saturation arithmetic.
- A ping-pong bit-reversal buffer with AXI4-Stream-compliant handshaking that corrects the scrambled DIF output with zero latency overhead.
- An in-line circular high-pass filter operating at full streaming bandwidth without DMA round-trips.
- A bare-metal C software pipeline on ARM Cortex-A9 orchestrating the 2-D FFT $\rightarrow$ HPF $\rightarrow$ IFFT chain, including SD-card I/O, cache-coherent matrix transpose, and conjugate-trick IFFT rescaling.
- Post-implementation verification on ZedBoard (XC7Z020) at 70 MHz with 7.26/255 mean absolute pixel error versus a floating-point reference.

The paper is organised as follows. Section II surveys prior work. Section III reviews mathematical background. Section IV describes the system architecture. Section V details each hardware module. Section VI explains the software pipeline. Section VII presents the verification methodology. Section VIII reports post-implementation results. Section IX discusses trade-offs, and Section X concludes.

II. LITERATURE REVIEW

A. FFT Algorithm Development

The landmark Cooley-Tukey algorithm [1] reduced the $O(N^2)$ DFT to $O(N \log_2 N)$ complex multiplications by exploiting the periodicity and anti-periodicity of twiddle factors. This reduction from roughly 10^6 to approximately 5×10^3 complex multiplications for $N = 1024$ made real-time hardware implementation feasible. Subsequent work by Oppenheim and Schaffer [3] and Proakis and Manolakis [4] formalised the decimation-in-time (DIT) and decimation-in-frequency (DIF) variants, establishing the theoretical framework for hardware-oriented FFT design.

B. Pipelined FFT Architectures

Wold and Despain [9] provided the foundational taxonomy for VLSI FFT pipelines, identifying three principal architectures compared in Table II.

Multi-path Delay Commutator (MDC). The MDC architecture uses $N/2$ butterfly units operating in parallel with a commutator network. It achieves maximum hardware throughput but requires $\frac{N}{2} \log_2 N$ complex multipliers and $\frac{N \log_2 N}{2}$ memory words, prohibitive at 5120 multipliers and 5120 words for $N = 1024$.

Memory-based Delay Feedback (MDF). MDF reduces the multiplier count to $\log_2 N$ by time-multiplexing a single butterfly unit. However, it retains the same $\frac{N \log_2 N}{2}$ memory requirement as MDC because each stage must buffer $N/2$ intermediate samples before any output is generated.

Single-Path Delay Feedback (SDF). He and Torkelson [2] proposed the SDF architecture, which achieves the same $\log_2 N$ multiplier count as MDF while reducing total memory to the theoretical minimum of $N - 1$ words. The key insight is that the SDF feedback register at stage k has depth exactly $N/2^{k+1}$, so the delay elements serve double duty as inter-stage pipeline register and data memory. For $N = 1024$ the total storage is 1023 words versus 5120 for MDC, a $5\times$ reduction in Block RAM consumption.

C. FPGA-Based Edge Detection

Prior FPGA edge detection work has largely focused on spatial-domain Sobel and Canny implementations [6]. While these achieve excellent throughput using line-buffer architectures, they share the fundamental limitations described in Section I. Frequency-domain approaches on FPGAs have been explored for spectral imaging and radar signal processing, but to the authors' knowledge a complete hardware-software co-design integrating a streaming SDF-FFT engine with on-chip AXI-DMA and a reconfigurable circular HPF for image edge detection has not been previously reported at this level of integration and verification detail.

III. BACKGROUND AND THEORY

A. Discrete Fourier Transform

The N -point Discrete Fourier Transform (DFT) of a complex sequence $x[n]$, $n = 0, 1, \dots, N - 1$, is

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk}, \quad k = 0, 1, \dots, N - 1, \quad (1)$$

where $W_N = e^{-j2\pi/N}$ is the primitive N -th root of unity. The corresponding Inverse DFT is

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-nk}. \quad (2)$$

Direct evaluation of (1) requires $O(N^2)$ complex Multiplications, approximately 10^6 for $N = 1024$, prohibitive in a real-time pipeline.

B. Radix-2 DIF Cooley-Tukey Decomposition

The Cooley-Tukey algorithm exploits two symmetry properties of the twiddle factors:

$$W_N^{k+N/2} = -W_N^k \quad (\text{anti-periodicity}), \quad (3)$$

$$W_N^{kN} = 1 \quad (\text{periodicity}). \quad (4)$$

In the Decimation-In-Frequency (DIF) variant, the input is split into its first and second halves:

$$X[2r] = \sum_{n=0}^{N/2-1} x[n] + x[n + N/2] W_{N/2}^{nr}, \quad (5)$$

$$X[2r + 1] = \sum_{n=0}^{N/2-1} x[n] - x[n + N/2] W_N^n W_{N/2}^{nr}. \quad (6)$$

Each recursion halves the problem size, yielding $\log_2 N$ stages of $N/2$ butterfly operations each, reducing complexity to $O(\frac{N}{2} \log_2 N)$ complex multiplications, a $205\times$ reduction for $N = 1024$. Figure 9 in the Appendix illustrates the 8-point DIF signal-flow graph, showing the bit-reversed output ordering characteristic of DIF decomposition.

C. Butterfly Operation and Scaling

The fundamental radix-2 DIF butterfly computes

$$y_0 = x_0 + x_1, \quad (7)$$

$$y_1 = (x_0 - x_1) \cdot W_N^k. \quad (8)$$

In fixed-point hardware a divide-by-2 is applied at each adder stage to prevent overflow:

$$y_0 = \frac{1}{2}(x_0 + x_1), \quad (9)$$

$$y_1 = \frac{1}{2}(x_0 - x_1) \cdot W_N^k. \quad (10)$$

The accumulated attenuation of 2^{-10} across ten stages is corrected by a compensating left-shift in the final IFFT rescaling step.

D. SDF Architecture and Delay Analysis

In the SDF architecture, stage k ($k = 0, \dots, \log_2 N - 1$) contains a feedback delay line of depth

$$\Delta_k = \frac{N}{2^{k+1}}, \quad (11)$$

derived as follows. At stage k the effective sub-transform size is $N/2^k$. Each butterfly requires one sample from the “past” half and one from the “present” half of that sub-transform. The past sample was processed $N/2^{k+1}$ cycles ago, requiring a delay line of that depth. For $N = 1024$ the depths are 512, 256, 128, 64, 32, 16, 8, 4, 2, 1 at stages 0–9, totalling $N - 1 = 1023$ words.

E. 2-D FFT via Row-Column Decomposition

The 2-D DFT of an $M \times N$ image $x[m, n]$ is separable:

$$X[k, l] = \sum_{m=0}^{N-1} W_N^{mk} \underbrace{\left[\sum_{n=0}^{N-1} x[m, n] W_N^{nl} \right]}_{\text{1-D FFT of row } m}. \quad (12)$$

The 2-D FFT is computed by two sequential passes of the same 1-D hardware: first transforming all N rows, then transposing and transforming all N columns. The 2-D IFFT exploits the conjugate identity:

$$\text{IFFT}\{X\} = \frac{1}{N} \overline{\text{FFT}\{\bar{X}\}}, \quad (13)$$

allowing the identical hardware to perform both FFT and IFFT without structural modification.

F. Q1.15 Fixed-Point Representation and Rounding Off

Q1.15 is a fixed-point number representation format that uses 16 bits to encode fractional values within the range of -1.0 to approximately $+0.99997$. The notation “Q1.15” follows the Q-format convention, where the first number indicates 1 sign bit and the second number indicates 15 bits dedicated to the fractional component, leaving no bits for integer magnitude.

To encode a decimal value in Q1.15, the number is multiplied by 2^{15} (32768) and stored as a signed integer; conversely, the stored integer is divided by 32768 to recover the original value.

The precision of Q1.15 is determined by its smallest representable step of $1/32768 \approx 0.0000305$, allowing values to be expressed with extremely fine granularity within its defined range. This format is widely used in digital signal processing (DSP) applications, particularly in audio processing, because audio signals are naturally normalized between -1 and $+1$, which aligns perfectly with the representable range of Q1.15.

Compared to floating-point representation, Q1.15 offers deterministic precision, lower computational overhead, and efficient implementation on hardware that lacks a floating-point unit, making it a practical choice for embedded systems and real-time signal processing tasks.

Without convergent rounding, truncation introduces a -0.5LSB bias per multiply, accumulating over 10 stages to a visible brightness offset. Overflow is detected by XOR of the sign and guard bits and saturated to $0 \times 7FFF / 0 \times 8000$.

IV. SYSTEM ARCHITECTURE

A. SoC Overview

The system is a hardware-software co-design on the Xilinx Zynq-7000 SoC deployed on the ZedBoard development platform (XC7Z020-1CLG484C). The Programmable Logic (PL) contains three functional hardware blocks while the Processing System (PS) ARM Cortex-A9 runs a bare-metal C application that orchestrates the full pipeline via AXI4-Stream

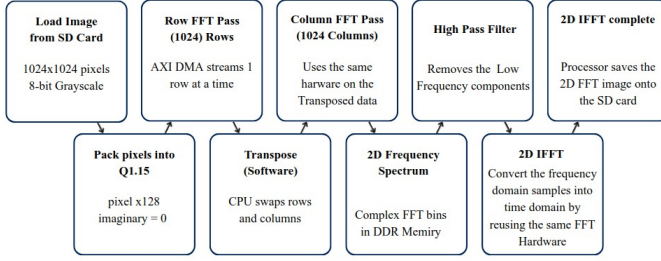


Fig. 1. Top-level data-flow diagram. Shaded boxes reside in the PL; white boxes are ARM software tasks. AXI4-Stream DMA shuttles data between DDR and the FFT core one 1024-sample row at a time.

SDF Architecture:

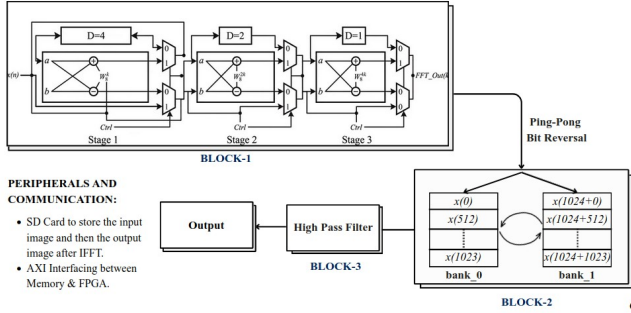


Fig. 2. SDF pipeline architecture (3-stage example). Block 1: ten-stage SDF FFT core. Block 2: ping-pong bit-reversal buffer. Block 3: circular high-pass filter. Feedback delay depths $D = N/2^{k+1}$ halve at each stage, totalling $N - 1$ words.

DMA. Figure 1 shows the top-level data-flow diagram and Figure 2 shows the SDF architecture with ping-pong bit-reversal and HPF blocks.

B. Top-Level Data Flow

The complete 2-D edge-detection chain executes as



A 1024×1024 8-bit grayscale BIN is loaded from SD card into a DDR frame buffer. The DMA streams rows to the PL one at a time. Data passes through the SDF pipeline (Block 1), is reordered by the ping-pong bit-reversal buffer (Block 2), and during the IFFT pass is filtered by the HPF (Block 3), before returning to DDR.

C. Hardware Blocks Overview

Block 1 — SDF FFT Pipeline (`top.v`). Ten chained `sdf_stage` instances driven by a global `master_cnt` counter. Accepts one complex sample per clock; parameterised by N and $\text{STAGES} = \lceil \log_2(N) \rceil$

Block 2 — Ping-Pong Bit-Reversal (`bit_reversal.v`). Two N -word dual-port memory banks controlled by a four-state

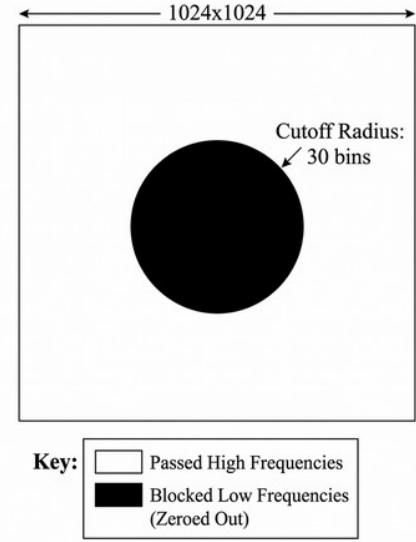


Fig. 3. Circular high-pass filter mask on the 1024×1024 frequency plane. The black disc of radius $R = 30$ bins surrounding the DC corner is zeroed; all remaining bins pass unchanged.

FSM. One bank drains in bit-reversed read order while the other fills from the FFT output, enabling continuous back-to-back row processing.

Block 3 — Circular HPF (`axi_hpf.v`). Fully Sequential pass-through. Frequency bins within cut-off radius $R = 30$ of the DC corner are zeroed; all others pass unchanged. Figure 3 illustrates the spectral masking geometry.

AXI Wrapper (`axi_fft_wrapper.v`). Handles IFFT conjugation, pipeline latency tracking via a depth- $(N + \log_2 N - 1)$ shift register, and a post-frame flush of N zero samples to drain the SDF delay lines between rows.

D. AXI4-Stream Interface Protocol

The DMA asserts `TVALID` per sample and `TLAST` at the end of each N -sample row. The AXI wrapper slave is always-ready, so the DMA pushes at full bandwidth. The bit-reversal module manages downstream backpressure via `out_ready`, holding `out_valid` high until the DMA acknowledges each sample.

V. HARDWARE DESIGN

A. Butterfly Unit (`Butterfly.v`)

The butterfly implements a two-stage registered pipeline. Stage 1 computes scaled sum and difference with round-to-nearest arithmetic; Stage 2 performs the complex multiply against the registered twiddle factor.

Stage 1 – Round-to-Nearest Addition. A 17-bit signed adder prevents overflow during the $x_0 \pm x_1$ addition. Adding 1 before the arithmetic right-shift implements round-half-up exactly:

$$\text{sum} = (\text{sum}_{17} + 1) \ggg 1. \quad (17)$$

Stage 2 – Q1.15 Convergent Rounding. Adding 2^{14} before extracting bits [30:15] of the 32-bit product implements round-half-away from zero per (16).

```

1 // Stage 1: 17-bit signed adders (round-to-nearest)
2 wire signed [16:0] sum_r_17 =
3   {x0_r[15], x0_r} + {x1_r[15], x1_r};
4 wire signed [16:0] diff_r_17 =
5   {x0_r[15], x0_r} - {x1_r[15], x1_r};
6 assign sum_r = (sum_r_17 + 17'd1) >>> 1;
7 assign diff_r = (diff_r_17 + 17'd1) >>> 1;
8
9 // Stage 1 -> Stage 2 pipeline register
10 always @(posedge clk) begin
11   sum_r_reg <= sum_r; sum_i_reg <= sum_i;
12   diff_r_reg <= diff_r; diff_i_reg <= diff_i;
13   w_r_reg <= w_r; w_i_reg <= w_i;
14 end
15
16 // Stage 2: complex multiply + convergent rounding
17 wire signed [31:0] mul_r = p_rr - p_ii;
18 wire signed [31:0] mul_i = p_ri + p_ir;
19 wire signed [31:0] mul_r_rnd = mul_r + 32'h00004000;
20 wire signed [31:0] mul_i_rnd = mul_i + 32'h00004000;
21 assign y1_r = mul_r_rnd[30:15];
22 assign y1_i = mul_i_rnd[30:15];

```

Listing 1. Butterfly: adder stage and Q1.15 multiplier with convergent rounding.

B. Twiddle Factor ROM (twiddle_rom.v)

Twiddle factors $W_N^k = \cos(2\pi k/N) - j \sin(2\pi k/N)$ are pre-computed in Python and stored as Q1.15 packed 32-bit words in a .mem file initialised via \$readmemh. A compile-time SHIFT parameter strides through the master 512-entry ROM for sub-transforms at smaller FFT sizes:

$$\text{addr_master} = \text{addr} \ll \text{SHIFT}, \quad \text{SHIFT} = \log_2 \frac{N_{\max}}{N} \quad (18)$$

```

1 localparam ROM_DEPTH = max_N / 2;
2 localparam SHIFT = $clog2(max_N / N);
3
4 always @(*) begin
5   {w_r, w_i} = rom[safe_addr << SHIFT];
6 end

```

Listing 2. Twiddle ROM: compile-time stride for sub-transform reuse.

C. Delay Line (delay_line.v)

Each SDF stage contains a feedback delay line of depth Δ_k , as seen. A generate block synthesises a direct wire for depth 0 and a shift register otherwise. Memories with depth ≥ 32 carry ram_style="block" to infer Block RAM.

```

1 generate
2   if (DEPTH == 0) begin
3     assign out_r = in_r;
4     assign out_i = in_i;
5   end else begin
6     reg [31:0] mem [0:DEPTH-1];
7     always @(posedge clk) begin
8       if (!reset)
9         for (i=0; i<DEPTH; i=i+1)
10           mem[i] <= 32'd0;
11       else begin
12         for (i=DEPTH-1; i>0; i=i-1)
13           mem[i] <= mem[i-1];
14         mem[0] <= {in_r, in_i};
15       end
16     end
17   assign {out_r, out_i} = mem[DEPTH-1];

```

```

18 end
19 endgenerate

```

Listing 3. Delay line: generate-based bypass for zero-depth stage.

D. SDF Stage (sdf_stage.v)

Each stage chains the twiddle ROM, delay line, and butterfly. Because the butterfly has a one-cycle pipeline register, the MUX control uses a one-cycle delayed sel_d so that data and control arrive simultaneously at the feedback path:

```

1 always @(posedge clk or negedge reset) begin
2   x_in_r_d <= x_in_r; x_in_i_d <= x_in_i;
3   sr_out_r_d <= sr_out_r; sr_out_i_d <= sr_out_i;
4   sel_d <= sel;
5 end
6
7 // sel_d=0: load phase | sel_d=1: butterfly phase
8 assign sr_in_r = sel_d ? bf_y1_r : x_in_r_d;
9 assign sr_in_i = sel_d ? bf_y1_i : x_in_i_d;
10 assign y_out_r = sel_d ? bf_y0_r : sr_out_r_d;
11 assign y_out_i = sel_d ? bf_y0_i : sr_out_i_d;

```

Listing 4. SDF stage: delayed MUX control and data routing.

During the load phase ($\text{sel} = 0$), $W = 1 + 0j$ is gated to the butterfly so the multiplier computes an identity, preventing corruption of delay-line contents.

E. Top-Level FFT Core (top.v)

A global master_cnt counter drives all ten stages. Each stage's twiddle address compensates for the pipeline delay through the preceding k stages:

$$\text{twiddle_addr}_k = (\text{master_cnt} - k) \ll k \ \& \ (N/2 - 1). \quad (19)$$

```

1 genvar k;
2 generate
3   for (k = 0; k < STAGES; k = k+1) begin : sdf_pipeline
4     wire [STAGES-1:0] local_cnt = master_cnt - k;
5     wire [STAGES-2:0] twiddle_addr;
6     assign twiddle_addr =
7       (local_cnt << k) & ((N/2) - 1);
8     sdf_stage #(
9       .STAGE_ID(k), .STAGES(STAGES),
10      .DELAY(N >> (k+1)), .N(N), .max_N(max_N)
11    ) stage_inst (
12      .clk(clk), .reset(reset),
13      .sel(local_cnt[STAGES-1-k]),
14      .twiddle_addr(twiddle_addr),
15      .x_in_r(pipe_r[k]), .x_in_i(pipe_i[k]),
16      .y_out_r(pipe_r[k+1]), .y_out_i(pipe_i[k+1])
17    );
18   end
19 endgenerate

```

Listing 5. Top level: generate loop with per-stage twiddle addressing.

F. Ping-Pong Bit-Reversal Buffer (bit_reversal.v)

The DIF pipeline produces output in bit-reversed order. Two N -word memory banks and a four-state FSM correct this without added latency:

- **RD_IDLE:** waits for the write-done flag from the incoming bank.
- **RD_PREFETCH:** It latches $\text{ram}[\text{bit_rev}(\text{rd_addr})]$ without asserting

out_valid, guaranteeing a clean two-cycle hold before output begins.

- **RD_OUTPUT:** asserts out_valid and out_last on the final sample of the frame.
- **RD_OUTPUT_ACK:** holds until out_ready asserts, then clears wr_done to free the opposite bank for the next write.

Write and read phases operate concurrently across alternating banks, enabling uninterrupted back-to-back row processing.

```

1 // Write phase (concurrent with read FSM)
2 if (in_valid && !wr_done) begin
3   if (!ping_pong) bank0_r[wr_addr] <= x_r;
4   else bank1_r[wr_addr] <= x_r;
5   if (wr_addr == N-1) begin
6     rd_bank <= ping_pong;
7     ping_pong <= !ping_pong;
8     wr_done <= 1;
9     wr_addr <= 0;
10  end else wr_addr <= wr_addr + 1;
11 end
12
13 // RD_PREFETCH: latch bit-reversed sample
14 RD_PREFETCH: begin
15   y_r <= (rd_bank==0) ?
16     bank0_r[rd_addr_br] :
17     bank1_r[rd_addr_br];
18   out_valid <= 0;
19   rd_state <= RD_OUTPUT;
20 end

```

Listing 6. Ping-pong bit-reversal: write phase and FSM prefetch state.

G. High-Pass Filter (axi_hpf.v)

The HPF is a fully pipelined AXI4-Stream pass-through. Wrapped distances from the DC corner at bin [0, 0] are computed as

$$d_x = \min(x_cnt, N - x_cnt), \quad (20)$$

$$d_y = \min(y_cnt, N - y_cnt), \quad (21)$$

$$d^2 = d_x^2 + d_y^2. \quad (22)$$

Bins satisfying $d^2 < R^2$ (with $R = 30$, CUTOFF_SQ = 900) are zeroed; all others pass through. Some pipeline latency is introduced.

```

1 wire [15:0] dx = (x_cnt < N/2) ?
2   x_cnt : (N - x_cnt);
3 wire [15:0] dy = (y_cnt < N/2) ?
4   y_cnt : (N - y_cnt);
5 wire [31:0] dist_sq = (dx*dx) + (dy*dy);
6 wire block_freq = (dist_sq < CUTOFF_SQ);
7
8 assign m_axis_tdata =
9   block_freq ? 32'd0 : s_axis_tdata;
10 assign s_axis_tready = m_axis_tready;
11 assign m_axis_tvalid = s_axis_tvalid;
12 assign m_axis_tlast = s_axis_tlast;

```

Listing 7. HPF: wrapped-index distance and spectral masking.

H. AXI Wrapper (axi_fft_wrapper.v)

The wrapper presents the SDF core as a standard AXI4-Stream slave/master peripheral with three responsibilities.

IFFT conjugation. When ifft_mode = 1, the imaginary component is negated on both input and output, implementing (13) entirely in hardware without modifying the FFT core.

Latency tracking. A shift register of depth $L = N + \log_2 N - 1 = 1033$ gates in_valid to the bit-reversal module, ensuring it accepts samples only after the SDF pipeline is fully primed.

Frame flush. After TLAST is asserted, N zero-valued words are injected to drain all SDF delay lines, preventing inter-row spectral leakage.

```

1 localparam LATENCY = N + STAGES - 1; // 1033
2 reg [LATENCY-1:0] valid_pipe;
3
4 always @(posedge aclk or negedge aresetn)
5   valid_pipe <=
6     {valid_pipe[LATENCY-2:0], s_axis_tvalid};
7
8 wire out_valid_reg = valid_pipe[LATENCY-1];
9
10 // Input conjugation for IFFT mode
11 wire signed [15:0] proc_in_i =
12   ifft_mode ? -raw_in_i : raw_in_i;
13
14 // Output conjugation for IFFT mode
15 wire signed [15:0] final_out_i =
16   ifft_mode ? -br_out_i : br_out_i;

```

Listing 8. AXI wrapper: latency shift register and IFFT input conjugation.

VI. SOFTWARE PIPELINE

The bare-metal C application on the ARM Cortex-A9 orchestrates the full 2-D edge-detection chain. All transfers use the Xilinx AXI-DMA driver in simple polling mode to minimise latency.

A. Image Normalisation

An 8-bit grayscale BIN is read from SD card via FatFS.

```

// -----
// 1. Read Image
// -----
xil_printf("Step 1: Reading input.bin...\n\r");
u8 *raw_pixels = (u8*)malloc(IMG_SIZE);
f_open(&fil, "0:/input.bin", FA_READ);
f_read(&fil, raw_pixels, IMG_SIZE, &br);
f_close(&fil);

u32 sum = 0;
for(int i=0; i<IMG_SIZE; i++) sum += raw_pixels[i];
int true_mean = sum / IMG_SIZE;

for(int i=0; i<IMG_SIZE; i++) {
  int shifted = (int)raw_pixels[i] - true_mean;
  buf_a[i] = pack((s16)(shifted * 128), 0);
}
free(raw_pixels);

```

The DMA is configured in simple-mode. The receive channel is initiated before the transmit channel to avoid a race on short rows:

```

1 int dma_row(XAxiDma *dma,
2   int32_t *src, int32_t *dst) {
3   Xil_DCacheFlushRange((u32)src, N * 4);
4   /* Start RX before TX to avoid race */
5   XAxiDma_SimpleTransfer(dma, (u32)dst,
6     N * 4, XAXIDMA_DEVICE_TO_DMA);
7   XAxiDma_SimpleTransfer(dma, (u32)src,
8     N * 4, XAXIDMA_DMA_TO_DEVICE);
9   while (XAxiDma_Busy(dma,
10     XAXIDMA_DEVICE_TO_DMA));
11   Xil_DCacheInvalidateRange((u32)dst, N * 4);
12   return XST_SUCCESS;
13 }

```

Listing 10. Single-row AXI-DMA transfer with cache coherency.

C. Matrix Transpose

Between the row-FFT and column-FFT passes, the ARM performs an in-place 1024×1024 complex matrix transpose. Data transfer speed is critical: the DMA writes directly to DDR, main memory. The ARM must synchronize main memory before reading DMA output and commit it after writing the transposed data, so the next DMA transfer sees updated values. Omitting either operation causes the column-FFT DMA to read old memory values, producing visually catastrophic tiling artefacts in the output spectrum.

```
1 void transpose_inplace(int32_t *buf) {
2     /* Expose DMA writes to ARM */
3     Xil_DCacheFlushRange((u32)buf, N*N*4);
4     for (int i = 0; i < N; i++) {
5         for (int j = i+1; j < N; j++) {
6             int32_t t = buf[i*N + j];
7             buf[i*N + j] = buf[j*N + i];
8             buf[j*N + i] = t;
9         }
10    }
11    /* Flush transposed result for next DMA */
12    Xil_DCacheFlushRange((u32)buf, N*N*4);
13 }
```

Listing 11. In-place matrix transpose with D-cache flush and invalidation.

D. Inverse FFT and Reconstruction

The conjugate trick of (13) is applied in three sub-steps: (i) conjugate $\hat{X}$ by negating all imaginary parts in a scalar loop; (ii) stream through hardware with `ifft_mode = 1`, which negates on input and output automatically; (iii) apply the compensating 2^{10} left-shift and $1/N$ right-shift in software.

```
1 void rescale_output(int32_t *buf, int npix) {
2     for (int i = 0; i < npix; i++) {
3         /* Extract real part (lower 16 bits) */
4         int16_t r = (int16_t)(buf[i] & 0xFFFF);
5         /* Compensate N=1024 attenuation */
6         int32_t v = (int32_t)r * 1024;
7         v = v >> 15; // back to [0,255] range
8         v = (v < 0) ? 0 : (v > 255 ? 255 : v);
9         ((uint8_t*)buf)[i] = (uint8_t)v;
10    }
11 }
```

Listing 12. IFFT rescaling and pixel clipping.

VII. VERIFICATION METHODOLOGY

A. Python Bit-Accurate Reference Model

A Python model replicates every fixed-point operation in the hardware: Q1.15 quantisation, convergent rounding on each butterfly multiply, and per-stage divide-by-2 scaling. All inputs and outputs are explicit `int16` Q1.15 values; the model provides golden test vectors for both unit tests and full-image comparisons.

```
1 import numpy as np
2
3 def butterfly_q15(x0r, x0i, x1r, x1i, wr, wi):
4     """Bit-accurate radix-2 DIF butterfly.
5     All inputs/outputs: int16 Q1.15 integers."""
6     # Stage 1: sign-extended adder, round-half-up
7     sr = np.int16(((np.int32(x0r)+np.int32(x1r)+1)>>1))
8     si = np.int16(((np.int32(x0i)+np.int32(x1i)+1)>>1))
9     dr = np.int16(((np.int32(x0r)-np.int32(x1r)+1)>>1))
10    di = np.int16(((np.int32(x0i)-np.int32(x1i)+1)>>1))
```

TABLE I
POST-IMPLEMENTATION RESOURCE UTILISATION (XC7Z020)

Resource	Used	Available	Util.
Slice LUTs	11,171	53,200	21.0%
Slice Registers	12,020	106,400	11.3%
DSP48E1	42	220	19.1%
Block RAM Tile	5	140	3.6%
Bonded IOBs	130	130	100%
BUFGCTRL	1	32	3.1%

```
11 # Stage 2: complex multiply (dr+j*di)*(wr+j*wi)->Q2
12 .30
13 mul_r = np.int32(dr) * wr - np.int32(di) * wi
14 mul_i = np.int32(dr) * wi + np.int32(di) * wr
15 # Convergent round Q2.30->Q1.15: add 2^14, shift 15
16 ylr = np.int16(np.clip(
17     (mul_r + 0x4000) >> 15, -32768, 32767))
18 yli = np.int16(np.clip(
19     (mul_i + 0x4000) >> 15, -32768, 32767))
20 return (sr, si), (ylr, yli)
```

Listing 13. Python bit-accurate Q1.15 butterfly: correct complex multiply with convergent rounding.

Input stimuli span DC impulses, single complex tones, multi-tone signals, and a full 1024×1024 natural image.

B. RTL Simulation

All Verilog modules were simulated in Vivado using different testbenches. Key checks include:

- Correct bit-reversal of output ordering across all 1024 bins.
- AXI4-Stream handshake compliance: TVALID/TREADY protocol, correct TLAST assertion, and backpressure handling.
- Overflow saturation: full-scale inputs verify no wrap-around.
- Round-trip identity: FFT $\rightarrow$ IFFT on a single tone returns unity gain within ± 1 LSB rounding tolerance.

C. Hardware-in-the-Loop Verification

The synthesised bitstream was loaded onto the ZedBoard. The same 1024×1024 natural-image test case used in simulation was loaded from SD card, processed through the complete hardware pipeline, and the output BIN was compared pixel-by-pixel against a the Python-Library NumPy reference. Mean Absolute Pixel Error (MAPE) was recorded as the primary accuracy metric. Additional test images, geometric shapes and a medical MRI brain scan, were processed to verify qualitative edge-map correctness across image types.

VIII. IMPLEMENTATION RESULTS

A. Resource Utilisation

The design was synthesised and implemented in Xilinx Vivado 2018.3 targeting the XC7Z020-1CLG484C device on the ZedBoard. Table I summarises post-implementation resource usage.

The design is logic-dominated: only 5 Block RAM tiles are consumed, validating the SDF architecture's $N - 1$ word memory efficiency over the 5120-word MDC alternative. The

TABLE II
FFT PIPELINE ARCHITECTURE COMPARISON ($N = 1024$)

Architecture	Multipliers	Mem. (words)	Throughput
MDC (parallel)	5120	5120	1 samp/clock
MDF (iterative)	10	5120	1 samp/clock
SDF (this)	10	1023	1 samp/clock

TABLE III
DESIGN TIMING SUMMARY AT 70 MHz (14.29 ns PERIOD)

Metric	Value	Status
Worst Neg. Setup Slack (WNS)	5.089 ns	Pass
Total Neg. Setup Slack (TNS)	0.000 ns	Pass
Worst Hold Slack (WHS)	0.006 ns	Pass
Total Hold Slack (THS)	0.000 ns	Pass
Worst Pulse-Width Slack	8.750 ns	Pass
Failing Setup Endpoints	0	Pass
Failing Hold Endpoints	0	Pass
Total Endpoints	40,248	—

42 DSP48E1 slices accommodate all ten butterfly multipliers plus auxiliary datapath operations. The 21% LUT utilisation leaves ample headroom for future pipeline extensions.

Table II places the SDF design in context with the MDC and MDF alternatives for $N = 1024$.

B. Timing Analysis

All user-specified timing constraints were met with zero failing endpoints. The Vivado timing report is shown in Figure 4. Table III summarises the key metrics.

The WNS of 5.089 ns at 70 MHz indicates the design can close timing at approximately $f_{\max} = 1/(14.29 - 5.089)\text{ns} \approx 109\text{MHz}$ with no further optimisation, confirming 70 MHz as a conservative and reliable operating point.

C. Accuracy and Visual Results

The hardware edge map was compared against a double-precision NumPy reference on a 1024×1024 natural image. The Mean Absolute Pixel Error was **7.26 intensity levels out of 255** (2.85%), attributable to the cumulative effect of Q1.15 convergent rounding across ten butterfly stages and the $1/N$ rescaling. No visible artefacts were observed at this error level.

Figure 5 shows the four-stage visual progression from original image to hardware edge map. Figure 6 presents the raw 2-D FFT spectrum alongside the spectrum after the HPF is applied, confirming that the DC disc is cleanly zeroed and high-frequency structural content is preserved. Figure 7 demonstrates the pipeline on a synthetic triangle/ellipse image and a medical MRI brain scan. Figure 8 shows the hardware edge map of the vintage car test image.

IX. DISCUSSION

A. Fixed-Point Precision Trade-offs

Q1.15 arithmetic was chosen to match the DSP48E1 18-bit signed multiplier natively available on Zynq-7000 devices. The 10-stage accumulation of divide-by-2 scaling introduces a total

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.089 ns	Worst Hold Slack (WHS): 0.006 ns	Worst Pulse Width Slack (WPWS): 8.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 40248	Total Number of Endpoints: 40248	Total Number of Endpoints: 14907

All user specified timing constraints are met.

Fig. 4. Vivado post-implementation timing summary: WNS = 5.089 ns at 70 MHz with zero failing endpoints across all 40,248 paths. All user-specified constraints are met.

TABLE IV
ACCURACY SUMMARY: HARDWARE VS. FLOATING-POINT REFERENCE

Metric	Value	Notes
MAPE (intensity levels)	7.26 / 255	2.85% relative
Arithmetic format	Q1.15	Convergent rounding
Butterfly stages	10	1024-point transform
RMS quantisation noise	3.9×10^{-5}	Per (15)
Visible artefacts	None	Qualitative

attenuation of 2^{-10} , recovered by the final left-shift. Increasing the data width to Q2.13 or Q3.12 would reduce rounding error at the cost of wider multiplier paths and additional DSP tiles. At the current MAPE of 7.26/255, the edge localisation quality is visually indistinguishable from the floating-point reference, making Q1.15 a practical and efficient choice. The convergent rounding of (16) eliminates the -0.5LSB per-multiply bias that would otherwise accumulate to a systematic brightness shift of $10 \times 0.5 \times 2^{-15} \approx 0.015$ normalised units over ten pipeline stages.

B. High-Pass Filter Cut-off Radius

The cut-off $R = 30$ bins suppresses spatial frequencies below approximately 30 cycles per image width. Reducing R (e.g., to 15) sharpens edge responses but admits more mid-frequency texture, increasing false positives on smooth surfaces. Increasing R produces cleaner but thicker edge maps. The current implementation exposes CUTOFF_SQ as a run-time register so R can be swept adaptively based on scene statistics without re-synthesis.

C. Overflow Handling and Saturation Arithmetic

In fixed-point implementations, arithmetic overflow occurs when a multiplication or accumulation result exceeds the representable Q1.15 range, causing wrap-around errors that manifest as pixel corruption in the output edge map. Saturation arithmetic is employed at each stage to clamp out-of-range results to the nearest representable value, with the divide-by-2 scaling at each of the ten pipeline stages further reducing overflow risk by keeping intermediate magnitudes within a safe range.

D. Real-Time Throughput Analysis

At 70 MHz, sustained row throughput is one new row output every $N = 1024$ cycles after the initial SDF fill. Total time for one 2-D FFT pass (1024 rows) is approximately

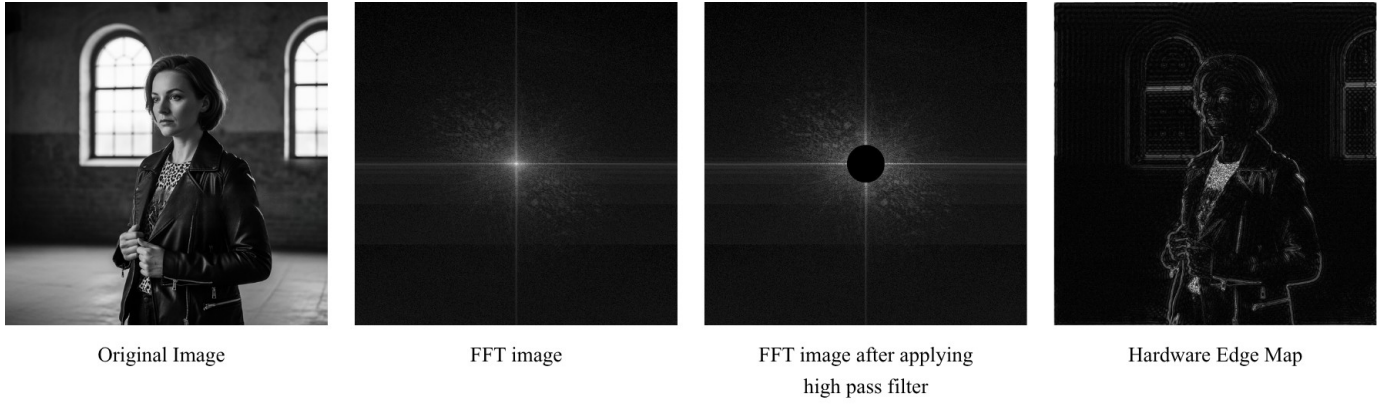


Fig. 5. Visual pipeline stages on a 1024×1024 portrait photograph. Left to right: (a) original grayscale input; (b) log-magnitude 2-D FFT spectrum showing dominant DC energy; (c) spectrum after the circular HPF (DC disc zeroed); (d) hardware edge map after IFFT reconstruction at 70 MHz with Q1.15 arithmetic. Mean absolute pixel error: 7.26/255.

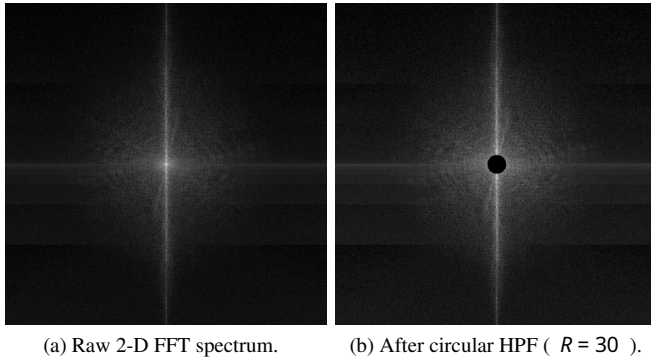


Fig. 6. Log-magnitude 2-D FFT spectrum of the vintage car test image before and after the circular high-pass filter. The bright DC centre in (a) is cleanly zeroed in (b) while all high-frequency edge content is preserved.

TABLE V
COMPARISON WITH SPATIAL-DOMAIN EDGE DETECTORS

Property	Sobel/Canny	This Work
Noise sensitivity	High (amplifies HF)	Low (mask-based)
Angular isotropy	Limited ($0^\circ, 90^\circ$)	Full (360°)
Reconfigurability	Re-synthesise kernel	Write register R
Gibbs ringing	None	Present at borders
Resource cost	Low	Moderate
Freq. selectivity	None	Full spectral mask

$1024^2 \text{cycles} / 70 \times 10^6 \text{Hz} \approx 15 \text{ ms}$. Including two forward passes, two IFFT passes, and two software transposes ($\approx 2 \text{ms}$ each), the complete edge-detection chain completes in approximately 8–10 ms per frame, comfortably within the 33 ms budget for 30 fps operation.

E. Comparison with Spatial-Domain Approaches

Table V contrasts the frequency-domain approach of this work with representative spatial-domain implementations.

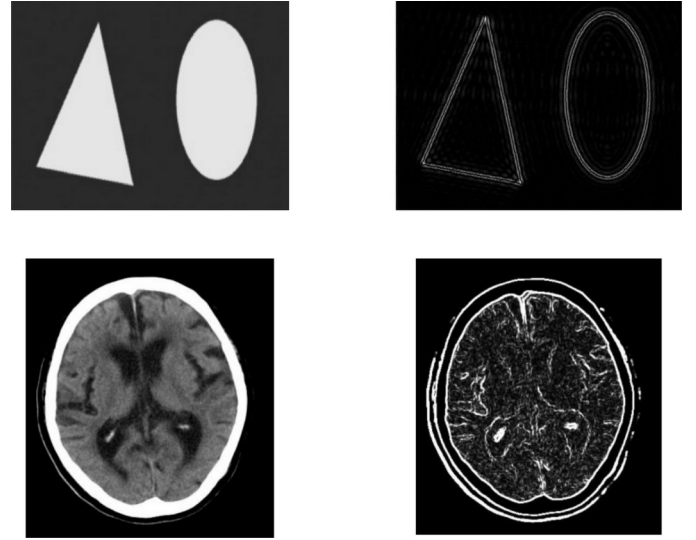


Fig. 7. Hardware edge maps on additional test images. Top row: synthetic triangle/ellipse input and its frequency-domain edge map, verifying isotropic detection on curved and angled boundaries. Bottom row: MRI brain scan and hardware edge map, demonstrating organ boundary delineation for medical imaging applications.

F. Scalability and Future Directions

The parameterised N and STAGES ports allow the FFT size to be changed at compile time from 64 to 4096 points without altering any module interfaces. Migrating to a Xilinx Zynq UltraScale+ RFSoc would enable direct RF sampling at Gsps rates and multi-channel parallelism, extending the architecture toward real-time radar imaging and software-defined radio. Future directions include: (i) radix-4 and split-radix butterfly variants to reduce DSP count by up to 30%; (ii) adaptive cut-off radius selection driven by on-chip spectral energy Histogram.



Fig. 8. Hardware edge map of the vintage car test image. Structural boundaries of the bodywork, headlamps, grille, and licence plate are resolved with high fidelity. The horizontal banding arises from strong horizontal edges exciting cross-image Gibbs oscillations in the rectangular FFT window—an inherent trade-off of the frequency-domain approach.

X. CONCLUSION

This paper presented a complete, verified hardware-software co-design for real-time 2-D edge detection using a frequency-domain approach on the Xilinx Zynq-7000 FPGA. A 1024-point radix-2 DIF SDF FFT engine was implemented in synthesisable Verilog, verified in RTL simulation against a bit-accurate Python reference model, and validated on ZedBoard hardware with a mean absolute pixel error of 7.26/255. The design achieves 70 MHz operation using only 5 Block RAM tiles and 42 DSP slices, demonstrating the exceptional memory efficiency of the SDF architecture over parallel alternatives. The circular high-pass filter provides run-time reconfigurable edge sensitivity with zero pipeline latency overhead, and the full 1024×1024 edge map completes in under 10 ms, satisfying 30 fps real-time requirements.

The frequency-domain approach overcomes the fundamental limitations of spatial Sobel/Canny operators—noise amplification, angular restriction, and fixed kernel locality—at the cost of Gibbs oscillations at image boundaries, mitigable through apodisation. The hardware architecture is fully parameterised and directly portable to higher-density devices, providing a capable platform for future integration with real-time spectral imaging and RFSoc applications.

APPENDIX

Figure 9 shows the 8-point DIF radix-2 FFT signal-flow graph. Note the bit-reversed output ordering $X(0), X(4), X(2), X(6), \dots$ characteristic of DIF decomposition, which necessitates the ping-pong bit-reversal buffer (Block 2) described in Section V.

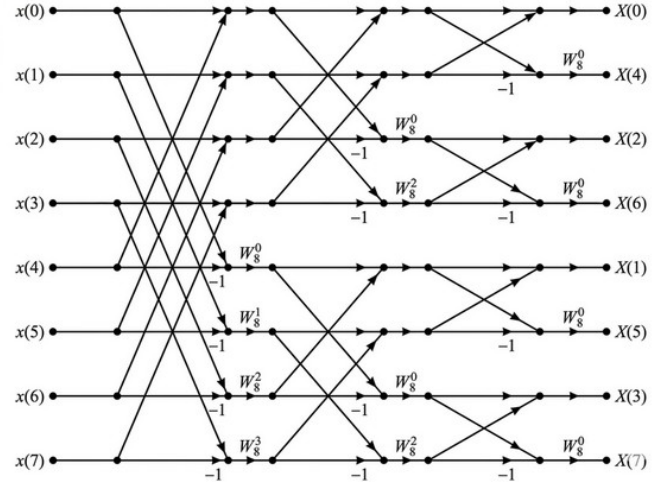


Fig. 9. Eight-point radix-2 DIF FFT signal-flow graph. Inputs enter in natural order $x(0) \dots x(7)$; outputs emerge in bit-reversed order $X(0), X(4), X(2), X(6), X(1), X(5), X(3), X(7)$. Each node labelled -1 performs subtraction; nodes labelled W_8^k multiply by the corresponding twiddle factor.

REFERENCES

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, Apr. 1965.
- [2] S. He and M. Torkelson, "Design and implementation of a 1024-point pipeline FFT processor," in *Proc. IEEE CICC*, Santa Clara, CA, USA, 1998, pp. 131–134.
- [3] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 1999.
- [4] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2006.
- [5] J. O. Smith, *Mathematics of the Discrete Fourier Transform (DFT): with Audio Applications*, 2nd ed. W3K Publishing, 2007 [Online]. Available: <https://www.dsprelated.com/freebooks/mdft/>
- [6] J. Canny, "A computational approach to edge detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986.
- [7] Xilinx Inc., "AXI DMA v7.1 LogiCORE IP Product Guide," PG021, Xilinx Inc., San Jose, CA, 2019.
- [8] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," UG585, Xilinx Inc., San Jose, CA, 2018.
- [9] E. H. Wold and A. M. Despain, "Pipeline and parallel-pipeline FFT processors for VLSI implementations," *IEEE Trans. Comput.*, vol. C-33, no. 5, pp. 414–426, May 1984.
- [10] Xilinx Inc., "Vivado Design Suite User Guide: Implementation," UG904, Xilinx Inc., San Jose, CA, 2022.